2025 10th International Conference on Communication, Image and Signal Processing (CCISP) | 979-8-3315-5278-7/25/$31.00 ©2025 IEEE | DOI: 10.1109/CCISP67522.2025.11282074

Nature-Inspired Metaheuristics for Wireless Sensor Network Coverage Optimization

1st Peiling Bai
Glasgow College, University of Electronic Science and Technology of China Hainan, China.
2022360902011@std.uestc.edu.cn

2nd Anlan Dong
Glasgow College, University of Electronic Science and Technology of China Chengdu, China, 11730.
2959908D@student.gla.ac.uk

3rd Zijin Zhou
Glasgow College, University of Electronic Science and Technology of China Chengdu, China, 11730.
2720773Z@student.gla.ac.uk

4th Xizhuo Wei
Glasgow College, University of Electronic Science and Technology of China Chengdu, China, 11730.
2960115W@student.gla.ac.uk

Amir Parnianifard
(IEEE Senior Member)
Glasgow College, University of Electronic Science and Technology of China, Chengdu, China, 11730.
Amir.Parnianifard@glasgow.ac.uk
**Corresponding author*

***Abstract*—Network coverage is a critical issue in wireless sensor networks (WSNs), which often operate in complex, hard-to-reach environments. Sensors are typically deployed randomly—such as via aerial distribution—resulting in uneven coverage with blind spots or redundant overlap. To overcome this, node placement must be dynamically optimized. However, meeting coverage and application constraints present an NP-hard optimization challenge, due to a large search space (|search space| = m^n, where m is the number of candidate positions per sensor, and n is the number of sensors). To address this, nature-inspired metaheuristic algorithms such as Genetic Algorithm (GA), Particle Swarm Optimization (PSO), Ant Colony Optimization (ACO), and Grey Wolf Optimizer (GWO) have been widely adopted in WSN tasks including localization, energy-efficient topology design, routing, and signal coverage. This paper focuses specifically on WSN coverage optimization and presents a systematic comparative study of four well-known nature-inspired algorithms: PSO, GWO, GA, and ACO. Their performance is assessed in terms of solution accuracy, robustness, and computational efficiency. The main contributions of this work are i) a comprehensive comparison of four popular nature-inspired metaheuristics applied to WSN coverage optimization; and ii) practical guidance for selecting the most suitable algorithm for large-scale problems involving 40 to 80 design variables representing sensor X and Y coordinates.**

Keywords*—Coverage Optimization, Wireless Sensor Network, Nature-Inspired Metaheuristics, Particle Swarm Optimization, Grey Wolf Optimizer, Genetic Algorithm, Ant Colony Optimization

I. INTRODUCTION

With the rapid development of wireless communication and Internet of Things (IoT) technology, Wireless Sensor Networks (WSNs) have greatly contributed to wireless connectivity and social networks [1]. As an integral part of IoT [2], WSNs are widely applied in diverse areas such as environmental monitoring and activity tracking, due to their minimal setup requirements and low preparation capacity [3]. Frequently, the challenge of ensuring adequate coverage while meeting particular application requirements often leads to NP-hard problems in optimization [4]-[6]. The more sensors that need to be installed, the larger the search space will be (|search space| = m^n, where m is the number of allowable locations for each sensor, and n is the number of sensors) [7]. This means that the complexity increases exponentially with the number of sensors deployed.

A significant number of researches have attempted to solve the coverage optimization problem, which has made good progress in the field [8]-[14]. In general, various optimization algorithms can be employed to tackle challenges in WSNs, including localization [15]-[18], energy-efficient topology [19], and shortest path to reduce both data collection time and sink travel expense [20], [21], and maximum signal coverage [22], [23]. Though, this paper limits its scope to common WSN coverage optimization problems [1], [24]. It aims to provide a comprehensive review of the optimization techniques employed in existing literature, further exploring and identifying the most effective optimization approaches applicable to addressing WSN coverage challenges.

To offer readers a clear and structured understanding of optimization techniques, a general overview is presented below. In general, optimization technologies are increasingly being utilized in engineering design to improve design quality and shorten the development cycles. Optimization involves the process of determining the optimal values for a set of design variables that meet one or more objectives while satisfying a predefined set of constraints. Most engineering design problems involve constraints, making it essential to employ robust methods capable of handling them effectively [25]-[28]. Generally, optimization techniques aimed at identifying optimal and feasible solutions under specific constraints can be categorized into model-based and metamodel-based (also known as surrogate-based) methods [29]-[36]. Classical model-based methods typically involve metaheuristic algorithms (also referred to as evolutionary algorithms), including Ant Colony Optimization (ACO) [11], Ant Lion Optimization (ALO) [12], Genetic algorithm (GA) [9], Particle Swarm Optimization (PSO) [10], Differential

979-8-3315-5278-7/25/$31.00 ©2025 IEEE

Authorized licensed use limited to: University of Pennsylvania. Downloaded on January 22,2026 at 04:59:01 UTC from IEEE Xplore. Restrictions apply.

Evolution (DE) [13], and more recent intelligence algorithms like the Grey Wolf Optimizer (GWO) [14]. These methods are widely adopted due to their simplicity, flexibility, derivative-free nature, and ability to escape local optima—attributes that make them appealing not only to computer scientists but also to researchers in a wide array of fields [14].

For computationally expensive optimization problems, such as computer-aided or simulation-based design, metamodel-based methods are often preferred [37]-[42]. These approaches combine surrogate models with metaheuristic algorithms to reduce computational cost while maintaining optimization accuracy [37]-[42]. However, the present study focuses on model-based methods, which have demonstrated considerable effectiveness in addressing WSN optimization problems [17], [43], [44].

In the context of WSNs, coverage rate is a key performance indicator, as it directly reflects the system's ability to monitor a target area. An effective node deployment strategy can significantly improve both energy efficiency and service quality in the network [45], [46]. Although sensor nodes are often randomly deployed in practice, this approach may lead to excessive node density, resulting in poor coverage and diminished monitoring performance [46], [47]. Thus, optimizing sensor placement is critical for maximizing coverage, particularly in large-scale networks, where determining the optimal layout poses a considerable challenge [48], [49].

To address the challenges of WSN coverage optimization, researchers have increasingly turned to metaheuristic algorithms, which have demonstrated significant success in improving WSN coverage. For instance, a modified version of the Fruit Fly Optimization Algorithm was proposed in [50] to enhance coverage. Liao et al. applied ACO for coverage expansion [51], while Ari et al. employed Bacterial Foraging Optimization for similar purposes. In [52], ACO was also used to optimize transmitter placement, resulting in improved average received signal power. Benchmark comparisons indicate that while Particle Swarm Optimization (PSO) and Genetic Algorithms (GA) tend to produce nearly identical antenna placement results, ACO generally requires a significantly higher number of objective function evaluations—suggesting a slower convergence rate. Among these techniques, GA has shown particular effectiveness in high-performance antenna design [7], [53], [54]. PSO has also been successfully applied to optimize base station placement for maximizing signal coverage [55]. Since its introduction by Mirjalili et al. [14], GWO has rapidly gained popularity in WSN optimization due to its robust capability in discovering optimal coverage configurations with high precision and efficiency [56].

The main highlights of this research include: i) comprehensive comparison of four widely recognized nature-inspired metaheuristics applied to WSN coverage optimization; ii) practical guidance for selecting the most appropriate algorithm for large-scale problems that involve 40 to 80 design variables representing the X and Y coordinates of sensors.

The remainder of this paper is structured as follows. Section 2 presents the fundamental methodology of each optimizer included in the current comparative study. Section 3 outlines the problem formulation related to coverage optimization in wireless sensor networks (WSNs). Section 4 presents the benchmark functions and simulation settings employed for the systematic comparison of algorithms. This section details the implementation procedures for the benchmark cases and provides a comprehensive statistical analysis of the comparative results. The discussion highlights the relative performance of each algorithm, offering insights into their strengths and limitations. Finally, Section 5 concludes the paper by summarizing the key findings and suggesting directions for future research.

II. Background Of Study

This section outlines the fundamental concepts and terminology of the four optimizers used in the comparative study: PSO, GWO, GA, and ACO.

A. Particle Swarm Optimization

The Particle Swarm Optimization (PSO) algorithm [14, 15] is a metaheuristic approach motivated by swarm intelligence theory and due to its simplicity and the requirement to tune only a small number of parameters leads to its strong optimization performance. It draws inspiration from the collective motion of bird flocks and fish shoals, as well as fundamental optimization concepts [16, 17].

Communication topology is a critical factor in the PSO optimization process. It governs the way particles exchange information, adjust their velocities, and collectively drive the swarm toward optimal solutions. In PSO, each particle updates its position by considering its individual best experience (personal best, p_{Best}) and the best position discovered either globally by the swarm or locally by its neighborhood (global or neighborhood best, g_{Best}). The optimization procedure begins with the random initialization of the particle swarm, assigning each particle as it is randomly initialized with position and velocity within the allowed area. Thereafter, a fitness function checks how good each particle's solution is at solving the problem, where the quality of the candidate solution in addressing the optimization objective is quantitatively evaluated.

In each iteration of PSO, every particle represents a potential solution and updates its position and velocity by tracking p_{Best} and g_{Best} found by the swarm.

$$v_{id}^{t+1} = v_{id}^{t} + c_1.rand(0,1).(p_{id}^{t} - x_{id}^{t}) + c_2.rand(0,1).\left(p_{gd}^{t} - x_{id}^{t}\right) \qquad (1)$$

$$x_{id}^{t+1} = x_{id}^{t} + v_{id}^{t}$$

In Eq.(1), the terms are defined as follows:

- v_{id}^{t}: Velocity of particle i in dimension d at t-th iteration.

Authorized licensed use limited to: University of Pennsylvania. Downloaded on January 22,2026 at 04:59:01 UTC from IEEE Xplore. Restrictions apply.

- x_{id}^{t}: Position of particle i in dimension d at t-th iteration.
- p_i: The optimal position discovered so far by particle i.
- p_g : The optimal position identified among the particles.
- c_1: A weighting parameter governs the influence of p_i on the particle's movement.
- c_2: A weighting parameter governs the influence of p_g on the particle's movement.
- $rand(0,1)$: Random number uniformly distributed between 0 and 1.

In PSO, exploration occurs when a particle's position significantly differs from its personal best (p_i) or the global best (p_g), driving it to search for new areas for better solutions. Conversely, when the difference is small, exploitation dominates, refining the particle's position around the best solutions. By balancing exploration and exploitation, PSO effectively combines global and local search strategies, offering a simple yet powerful approach for solving diverse optimization problems.

B. Grey Wolf Optimizer

Mirjalili et al. [14] introduced the GWO constitutes a notable breakthrough in the domain of swarm intelligence. Motivated by the social stratification and collaborative hunting strategies of grey wolves, the Grey Wolf Optimizer (GWO) simulates a population of candidate solutions using a well-defined hierarchical framework.

- The alpha (α) represents the best solution, leading the hunt and guiding the search.
- The beta (β) and delta (δ) wolves, the second and third-best solutions, assist the alpha in decision-making.
- The remaining solutions are designated as omega (ω) wolves, adhering to the decisions and movements of the dominant members.

Due to its hierarchical structure, GWO could effectively balance between global search and local refinement by mimicking grey wolves' hunting actions, such as tracking and encircling. The encircling strategy is mathematically represented by the following equation:

$$\vec{M}(t+1) = \vec{M}_p(t) - \vec{Q}.|\vec{H}.\vec{M}_p(t) - \vec{M}(t)| \tag{2}$$

where:

- t : current iteration
- $\vec{M}_p$: prey position (target solution)
- $\vec{M}$: wolf position
- $\vec{Q}$ and $\vec{H}$: coefficient vectors, defined as:

$$\vec{Q} = 2\vec{a}.\vec{r}_1 - \vec{a} \quad \text{and} \quad \vec{H} = 2\vec{r}_2 \tag{3}$$

Here, r_1 and r_2 are random vectors uniformly distributed between 0 and 1, and a decreases linearly from two to zero as the iterations progress, following the equation:

$$\vec{a}(t) = 2 - \frac{2t}{MaxIter} \tag{4}$$

This gradual reduction promotes exploitation by reducing randomness over time, thereby steering the wolves toward the optimal solution. The position of each wolf is updated based on the alpha, beta, and delta wolves, which correspond to the optimized solutions. The updated position is calculated using the following formula:

$$\begin{aligned}\vec{M}(t+1) = \tfrac{1}{3}\big(&\vec{M}_\alpha(t) - \vec{Q}_1.\left|\vec{H}_1.\vec{M}_\alpha(t) - \vec{M}(t)\right| + \\ &\vec{M}_\beta(t) - \vec{Q}_2.\left|\vec{H}_2.\vec{M}_\beta(t) - \vec{M}(t)\right| + \vec{M}_\delta(t) - \\ &\vec{Q}_3.\left|\vec{H}_3.\vec{M}_\delta(t) - \vec{M}(t)\right|\big)\end{aligned} \tag{5}$$

This hierarchical update strategy directs the search process based on the leading solutions, allowing GWO to efficiently balance exploration and exploitation of the search space, thereby achieving strong performance across various optimization challenges.

C. Genetic Algorithm

The GA [57], first introduced by John Holland, is a powerful stochastic optimization method motivated by the principles of natural selection and genetics.

Compared with conventional search techniques, Genetic Algorithm (GA) initializes with a population of randomly generated candidate solutions, referred to as a chromosome, represents a set of potential solutions encoded as binary strings representing genes.

$$\boldsymbol{(b_j - a_j) \times 10^t \leq 2^{l_j} - 1} \tag{6}$$

- $a_j \leq x_j \leq b_j$: defines the range of the variable x_j;
- t: the number of decimal places required for precision;
- $l_j$the smallest integer that satisfies the inequality above.

The population evolves over successive iterations, known as generations. In each generation, chromosomes are evaluated based on a predefined fitness function.

$$\boldsymbol{p_i = \frac{f\ (x_i)}{\sum_{k=1}^{n} f\ (x_k)}} \tag{7}$$

- f（x_i）: the fitness value of individual i
- n: the population size

Authorized licensed use limited to: University of Pennsylvania. Downloaded on January 22,2026 at 04:59:01 UTC from IEEE Xplore. Restrictions apply.

Offspring are produced through crossover between two parents or mutation of a single chromosome. The next generation is selected from both parents and offspring based on fitness, keeping the population size unchanged.

$$\boldsymbol{p_m(t) = p_{m,\ initial} \cdot e^{-\alpha t/T}} \tag{8}$$

- α : the attenuation coefficient
- T: the maximum number of iterations

Chromosomes with superior fitness are more likely to be chosen, leading the algorithm over time to converge toward the best solution or an approximation thereof.

D. Ant Colony Optimizer

ACO algorithm is inspired by the collective foraging behavior observed in real ants. Initially, the search begins by randomly selecting a feasible point within the search space to serve as the starting nest for the first colony. A predetermined number of ants are then assigned to each colony. Each ant commences its exploration from the nest, seeking and exploiting new feasible points by traversing the search space, guided either by pheromone trails or by random movements with a variable step size [58].

To balance exploration and exploitation, direction selection follows an ε-greedy-like strategy. Specifically, an ant generally chooses the most promising direction stored in its memory.

In this study, the probabilistic direction selection closely emulates the natural behavior of real ants, wherein movement decisions are influenced by both the strength of pheromone trails and heuristic information. The combined data regarding pheromone intensity and heuristic desirability for each possible direction are stored in the Ant Routing Table (ART). The selection probability (p_s) for each direction can be calculated using the equation below:

$$\boldsymbol{b_i = \frac{\alpha_i \times \beta_i^{\gamma}}{\sum_{j=1}^{Directions} \alpha_j \times \beta_j^{\gamma}}} \tag{9}$$

$$\forall \boldsymbol{i} \in \{\boldsymbol{Directions}\}, \qquad \boldsymbol{where\ \gamma = 0.4}$$

$$\boldsymbol{p_s = b_i / \left(\sum_{j=1}^{Directions} b_j \right)}$$

- b_i : Decision value for direction i , combining pheromone intensity and heuristic information.
- α_i: Pheromone trail strength for direction i.
- β_i : Heuristic value for direction i , indicating its desirability.
- γ: Parameter controlling the influence of heuristic information $\gamma = 0.4$.

Variable step size and memorized or random directions are two main factors for the increment movement of ants.

$$\boldsymbol{\overrightarrow{X_{J+1}} = \overrightarrow{X_J} + r_a \cdot \vec{\pi}(\cdot)} \tag{10}$$

- $\overrightarrow{X_J}$: The vector representing the position or state of the ant at Jth iteration
- r_a: The radius of activity, which controls the maximum possible step size or the exploration range for movement at the current iteration.
- $\vec{\pi}(\cdot)$: The direction of movement.

A search radius is assigned to each colony, which gradually decreases as the algorithm progresses. Consequently, colonies initialized at later stages operate with a smaller search radius. Additionally, within each colony, the search radius diminishes over time as individual ants conduct their search. All ants begin their exploration using the initial radius defined for their respective colony and terminate their search once the radius is reduced to zero.

III. Statement Of WSN Coverage Optimization

Each sensor offers a certain circular covering area. To determine whether one location is contained in the sensing area, the space between this target location and the sensor is calculated and then compared with sensor's radius.

The whole monitoring space of WSN is a 2D area which is separated to $P \times Q$ pixels. To measure the coverage rate of the system, the total number of pixels covered in the sensing area is measured as x and $x/(P \times Q)$ will be the coverage rate. Apart from the above, the boundary effect is ignored for this sensing system, and every sensor node is identical with the same monitoring area.

Define the perceived sensor's radius in the monitoring space as R and the set of N sensor nodes as $T = \{t_1, t_2, \ldots, t_N\}$ where t_k denotes the coordinate of the k-th sensor, (x_k, y_k). The coordinates of f-th points which need monitored and are called the monitoring nodes, is h=(x_f, y_f). The space between the monitoring point$h$ and sensor t_k is computed in equation11:

$$d(t_k - h) = \left(\left(x_k - x_f\right)^2 + \left(y_k - y_f\right)^2 \right)^{\frac{1}{2}} \tag{11}$$

This model adopts a binary detection strategy, meaning a point h is regarded as sensed by a sensor t_k if it falls within the coverage radius. The corresponding detection probability is expressed as:

$$p(t_k - h) = \begin{cases} 1, & d(t_k - h) \le 0 \\ 0, & d(t_k - h) > 0 \end{cases} \tag{12}$$

If $p(t_k - h)$ equals to 1, then the monitoring point h is successfully covered by sensor t_k. Conversely, if it equals 0, the points h is not covered by sensor t_k.

Given that it is possible for point in monitoring space is monitored by several sensor nodes, the possibility of this point h is monitored by node set T is calculated by:

Authorized licensed use limited to: University of Pennsylvania. Downloaded on January 22,2026 at 04:59:01 UTC from IEEE Xplore. Restrictions apply.

$$p(T,h) = 1 - \prod_{t_k \in T} \left(1 - \left(p(t_k - h)\right)\right) \quad (13)$$

To quantify the overall coverage efficiency of the sensor set T, the percentage of the monitored area covered is given by:

$$R_c = \frac{\sum_{h \in P \times Q} p(T,h)}{P \times \mathrm{Q}} \times 100 \quad (14)$$

In this paper, aiming at testing optimizers for a more complex area, it is assumed that there are some obstacles over the design space $P \times Q$, here assume 100×100 2D map of the test area. We also assume that obstacle areas are not counted into effective monitoring space, and just $P \times Q - obstacles$ are aimed to be covered. Therefore, Eq. (14) is modified to:

$$R_c = \frac{\sum_{h \in Pf \times Qf} p(T,h)}{Pf \times Qf} \times 100 \quad (15)$$

where $Pf \times Qf$ are pixels coordinates of the free area in 2D area which excludes pixels of obstacles' areas.

IV. Comparative Study

A. Simulation Setup

In this research, we created an optimization model for enhancing the coverage rate of WSN over a 2D area with obstacles. By applying PSO, GWO, GA, and DE, this research investigates how metaheuristic methods can be utilized to determine sensor placement strategies that result in improved monitoring coverage across the given environment. Our design assumes that sensor node cannot locate in the obstacles and each of them are homogeneous with identical sensing radius. The external disturbance is also negligible.

The decision variable in this design is the coordinates of each sensing node. These sensor positions are encoded as a sensor location vector:

$$SenseLoc = \{x_1, y_1, x_2, y_2, \ldots, x_n, y_n\} \quad (16)$$

where each coordinate pair (x_n, y_n) denotes the position of i^{th}sensor in the 100×100 pixel grins of the 2D monitoring area. Since the coordinates variables are defined within [0,100] and constrained by obstacles which means sensors cannot be in the obstacles, the search space of them is defined as a set of pixel points outside obstacles. The total number of the decision variable is determined by the number of sensors which is defined by us.

This optimization is based on binary perception model, which means a pixel is covered when its Euclidean distance to any sensor is within the predefined sensing radius. The objective function is formulated as the negative ratio of the covered pixels, which means pixels outside the obstacles, covered by at least one sensor node. The negativity is designed for minimization optimization.

$$\text{objective function} = -\left(\frac{N_{covered}}{N_{feasible}} \times 100\right) \quad (17)$$

where $N_{covered}$ quantifies the covered pixels, whereas $N_{feasible}$ defines overall pixel number eligible for coverage.

In this paper, a total of four cases study are conducted with different quantities of sensor nodes and sensing radius to evaluate four algorithms under varying levels of complexity: case 1 deploys 20 sensor nodes, each with a sensing radius of 5 meters. Case 2 also uses 20 nodes but increases the coverage radius to 10 meters. Case 3 scales the network to 40 sensors while maintaining a 5-meter sensing range. Case 4 uses 40 sensors, each with a 10-meter radius.

In each case, four algorithms are set with identical maximal number of iterations which are 10, variable bounds, and shared search space constraints. Each of algorithms are executed three times to access the robustness given to the stochastic nature of metaheuristics.

The performance of algorithms will be assessed in the aspects of average coverage rate, standard deviation, maximal and minimal coverage rate and total computational time. Above metrics will be analyzed to balance accuracy, convergence and computational resources. The whole stimulation is conducted in MATLAB, and the obstacle map is dynamically generated for each stimulation.

B. Results and Discussion

1) Results for Four Cases: The performance of PSO, GWO, GA, and ACO was evaluated across four cases in a 100×100-pixel grid with obstacles, each involving 10 runs to optimize wireless sensor network (WSN) coverage.

a) Case 1: 20 Sensors, 5-meter Coverage Radius

TABLE summarizes the performance metrics for Case 1. The average coverage rate ranged from 16.388% (ACO) to 18.949% (GA), with GA performing best. GA also achieved the highest maximum coverage at 19.371%, while ACO had the lowest at 16.856%. The coverage distribution is shown in Figure 1. In the figure, GA's strategic sensor placement around obstacles maximizes overlap, minimizing uncovered areas. ACO, however, exhibits larger gaps, especially near the grid's center and edges. PSO and GWO show moderate overlap with some uneven distribution.

For robustness, GA showed the highest stability with a standard deviation of 0.33777, as seen in its tight coverage trend (Figure 5, green triangles) and concentrated distribution (Figure 6). PSO had the highest variability with a standard deviation of 0.75193.

Authorized licensed use limited to: University of Pennsylvania. Downloaded on January 22,2026 at 04:59:01 UTC from IEEE Xplore. Restrictions apply.

TABLE 1.: PERFORMANCE METRICS FOR CASE 1

Method	Ave	Std	Max	Min	Time Elapsed(s)
PSO	16.887	0.75193	17.768	15.401	55.152
GWO	17.434	0.62848	18.816	16.523	53.036
GA	18.949	0.33777	19.371	18.434	60.498
ACO	16.388	0.6322	16.856	15.043	62.016

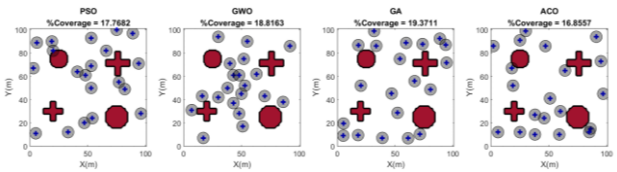

Fig. 1. Sensor Coverage Optimization for 20 Sensors with 5-meter Radius Using PSO, GWO, GA, and ACO (Gray circles: sensor coverage; Blue crosses: sensors; Red polygons & Red crosses: obstacles).

b) Case 2: 20 Sensors, 10-meter Coverage Radius

Case 2, involving 20 sensors with a 10-meter coverage radius. Similarly, GA led with an average coverage of 56.292% (max: 57.657%), and ACO trailed at 47.798% (max: 50.74%). GA was the most consistent (std: 1.0053), while ACO had the highest variability (std: 2.6263). Runtimes spanned 55.172s (GWO) to 66.002s (ACO). The coverage distribution is illustrated in Figure 2. The figure highlights GA's extensive overlap, covering most of the grid with minimal gaps. ACO's coverage is patchy, with noticeable uncovered areas around the center and edges. PSO and GWO improve over Case 1 but still leave some gaps near the grid's boundaries.

TABLE 2. PERFORMANCE METRICS FOR CASE 2

Method	Ave	Std	Max	Min	Time Elapsed(s)
PSO	51.035	1.9272	54.217	48.619	54.168
GWO	52.541	2.4059	58.656	50.21	55.006
GA	56.292	1.0053	57.657	54.599	57.184
ACO	47.798	2.6263	50.74	42.676	62.467

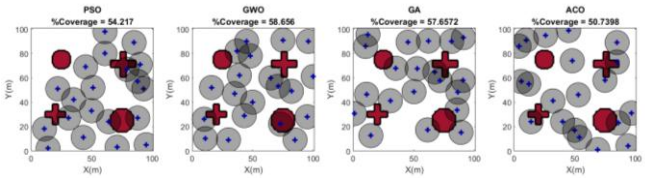

Figure 2. Sensor Coverage Optimization for 20 Sensors with 10-meter Radius Using PSO, GWO, GA, and ACO (Gray circles: sensor coverage; Blue crosses: sensors; Red polygons & Red crosses: obstacles).

c) Case 3: 40 Sensors, 5-meter Coverage Radius

As shown in Table 3, GA continued to excel with an average coverage of 34.015% (max: 35.684%), against ACO's lowest at 28.97% (max: 31.677%). Unlike previous cases, PSO was the most stable (std: 0.46516), while ACO showed the highest variability (std: 1.5627). Runtimes ranged from 221.31s (PSO) to 257.27s (ACO). The coverage distribution is depicted in Figure 3. In the figure, GA's dense sensor packing around obstacles ensures high coverage with minimal gaps. ACO struggles with visible gaps near the center and edges, reflecting its lower performance. PSO and GWO achieve more uniform distribution than in Case 1, though some uncovered areas persist. Moreover, there are some sensors on obstacles for PSO and ACO, impacting coverage.

TABLE 3. PERFORMANCE METRICS FOR CASE 3

Method	Ave	Std	Max	Min	Time Elapsed(s)
PSO	30.217	0.46516	31.085	29.519	221.31
GWO	30.401	0.77823	31.911	29.285	231.23
GA	34.015	0.76894	35.684	33.12	246.52
ACO	28.97	1.5627	31.677	27.053	257.27

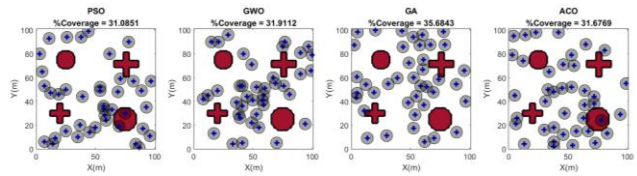

Figure 3. Sensor Coverage Optimization for 40 Sensors with 5-meter Radius Using PSO, GWO, GA, and ACO (Gray circles: sensor coverage; Blue crosses: sensors; Red polygons & Red crosses: obstacles).

d) Case 4: 40 Sensors, 10-meter Coverage Radius

Similarly, GA achieved the highest average coverage at 83.568% (max: 86.461%), with ACO at 72.694% (max: 77.374%). GA and PSO exhibited similar consistency (std: 1.2354 and 1.2406), while ACO was the least stable (std: 2.9083). Runtimes varied from 217.32s (GWO) to 254.36s (ACO). The coverage distribution is presented in Figure 4. The figure shows GA's near-complete coverage, with sensors tightly packed and overlapping across the grid. ACO leaves noticeable gaps near obstacles and edges, indicating less effective placement. PSO and GWO perform well but have minor uncovered areas near the grid's boundaries.

TABLE 4. PERFORMANCE METRICS FOR CASE 2

Method	Ave	Std	Max	Min	Time Elapsed(s)
PSO	76.433	1.2406	78.274	73.97	225.33
GWO	76.079	1.7031	78.742	73.748	217.32
GA	83.568	1.2354	86.461	82.121	234.82
ACO	72.694	2.9083	77.374	66.4	254.36

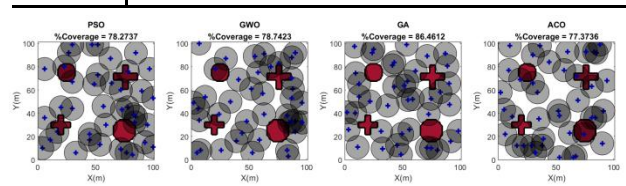

Figure 4. Sensor Coverage Optimization for 40 Sensors with 10-meter Radius Using PSO, GWO, GA, and ACO (Gray circles: sensor coverage; Blue crosses: sensors; Red polygons & Red crosses: obstacles).

Authorized licensed use limited to: University of Pennsylvania. Downloaded on January 22,2026 at 04:59:01 UTC from IEEE Xplore. Restrictions apply.

2) Comparative Discussion: This section compares the performance of PSO, GWO, GA, and ACO across four cases, focusing on coverage rate, robustness, runtime, as supported by numerical data and visualizations.

a) Coverage Rate Comparison: GA consistently outperformed other algorithms in average coverage rate across all cases, while ACO performed the worst. Figures 1–5 visually confirm this trend, with GA showing extensive overlap and minimal gaps, whereas ACO exhibits larger uncovered areas. GA's coverage maps highlight strategic sensor placement. ACO's coverage suffers more due to frequent sensor misplacement, this exacerbates gaps.

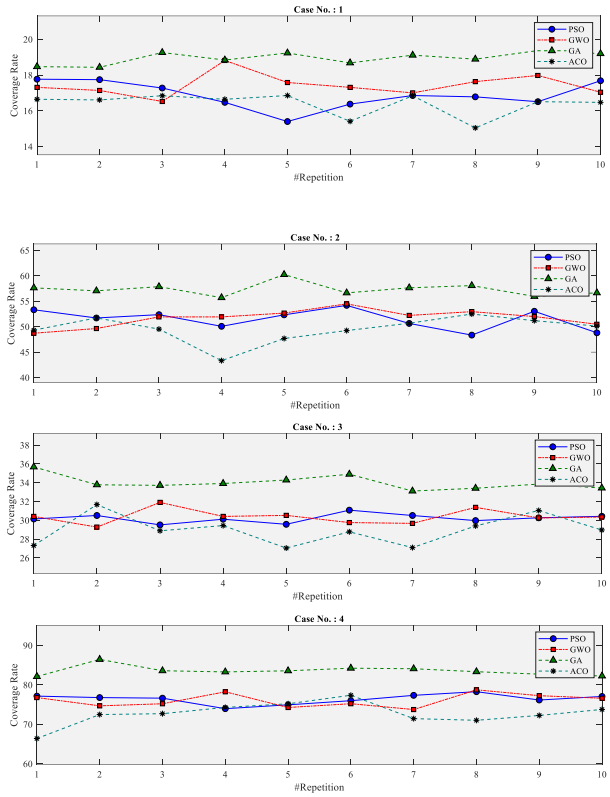

Figure 5. Coverage Rate Trends Over Iterations

b) Robustness and Stability: Robustness, measured by the standard deviation of coverage rates over 10 runs, varied across cases. According to Table 1-4, in Cases 1 and 2, GA was the most stable, while PSO and ACO showed the highest variability, respectively. In Case 3, PSO emerged as the most stable, with ACO having the highest variability. In Case 4, GA and PSO exhibited similar consistency, while ACO was the least stable. Figures 5 also illustrate these trends through coverage rate trends over iterations: GA's green line shows minimal fluctuations, while ACO's cyan line displays significant variability, especially in Case 3.

As shown in Figure 6, GA's coverage rate leads across all cases, consistently achieving the highest values. However, in Case 3, GA's data points are relatively more dispersed compared to PSO, whose blue line shows tighter clustering around the mean, indicating PSO's greater concentration in that case.

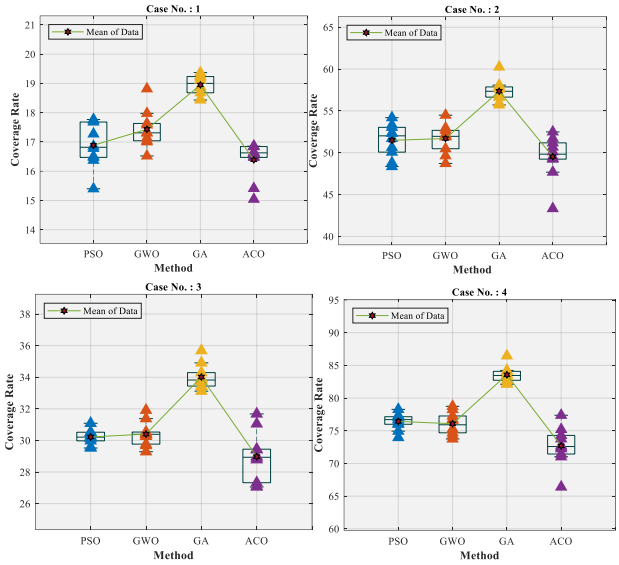

Figure 6. Coverage rates across 10 runs, with the mean marked by a star.

c) Runtime Analysis: Runtime increased with the number of sensors, reflecting higher computational complexity. Figure 7 provide runtime comparisons, showing ACO consistently taking the longest, while PSO and GWO are the fastest. GA falls in between, with its runtime reflecting a trade-off for its superior coverage optimization.

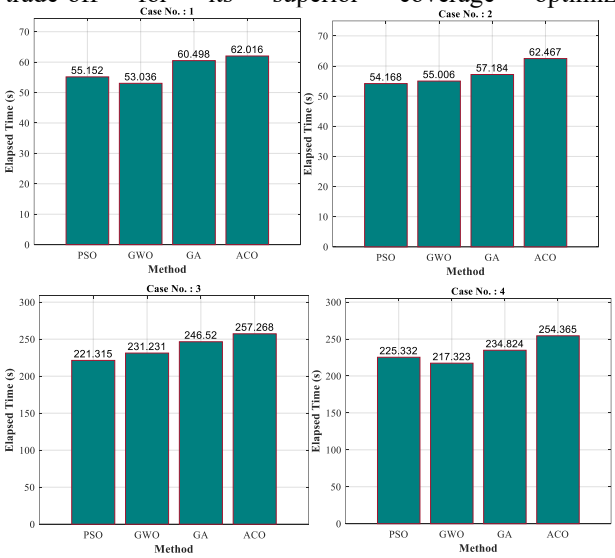

Figure 7. Runtime across 4 cases.

Overall, GA proves to be the most effective algorithm, achieving the highest coverage rates across all cases, though its runtime is higher than GWO's. ACO consistently underperforms, with the lowest coverage, highest variability, and longest runtimes, further impacted by frequent sensor misplacement on obstacles. PSO and GWO offer a balanced performance, with PSO showing notable stability in Case 3 and GWO providing the fastest runtimes. The visual data

Authorized licensed use limited to: University of Pennsylvania. Downloaded on January 22,2026 at 04:59:01 UTC from IEEE Xplore. Restrictions apply.

(Figures 1–4), iteration trends (Figures 5), and scatter plots (Figures 6) underscore GA's superior optimization, but all algorithms would benefit from enhanced obstacle avoidance to prevent sensor misplacement, which notably impacts coverage efficiency.

V. Conclusion

This paper compares four nature-inspired metaheuristic algorithms—PSO, GWO, GA, and ACO—for optimizing sensor node placement in wireless sensor networks (WSNs) to maximize coverage. Their performance was evaluated based on solution accuracy, robustness (using mean, standard deviation, and extremes of coverage), and computational efficiency across four case studies with varying sensor counts and sensing radii. Despite its longer runtime, GA consistently achieved the highest average coverage and lowest variability, making it the most effective and reliable algorithm for high-accuracy applications. While PSO and GWO ran faster, they showed lower and more variable coverage, and ACO performed the worst in both accuracy and speed. Overall, GA is recommended for precision-focused WSN deployment, with future work suggested to explore additional algorithms and more complex deployment scenarios.

References

[1] A. Parnianifard, S. Mumtaz, S. Chaudhary, M. A. Imran, and L. Wuttisittikulkij, "A data driven approach in less expensive robust transmitting coverage and power optimization," Sci. Rep., vol. 12, p. 17725, 2022.

[2] X. Qi, Z. Li, C. Chen, and L. Liu, "A wireless sensor node deployment scheme based on embedded virtual force resampling particle swarm optimization algorithm," Appl. Intell., vol. 52, no. 7, pp. 7420–7441, 2022.

[3] S. Meguerdichian, F. Koushanfar, M. Potkonjak, and M. B. Srivastava, "Coverage problems in wireless ad-hoc sensor networks," in Proc. IEEE INFOCOM 2001, vol. 3, 2001, pp. 1380–1387.

[4] A. Duttagupta, A. Bishnu, and I. Sengupta, "Maximal breach in wireless sensor networks: Geometric characterization and algorithms," in Int. Symp. Algorithms Exp. Sensor Syst., Wireless Netw. Distrib. Robotics, 2007, pp. 126–137.

[5] A. Gogu, D. Nace, A. Dilo, and N. Mertnia, "Optimization problems in wireless sensor networks," in Proc. Int. Conf. Complex, Intell., Softw. Intensive Syst., 2011, pp. 302–309.

[6] O. Zorlu and Ö. K. Şahıngöz, "Node placement with evolutionary algorithms for maximum coverage of heterogeneous WSNs," in Proc. 25th Signal Process. Commun. Appl. Conf. (SIU), 2017, pp. 1–4.

[7] A. Jauhri, J. D. Lohn, and D. S. Linden, "A comparison of antenna placement algorithms," in GECCO 2014 Companion, 2014, pp. 1223–1230.

[8] R. V. Kulkarni and G. K. Venayagamoorthy, "Particle swarm optimization in wireless-sensor networks: A brief survey," IEEE Trans. Syst., Man, Cybern. C, vol. 41, no. 2, pp. 262–267, 2010.

[9] D. Whitley, "A genetic algorithm tutorial," Stat. Comput., vol. 4, no. 2, pp. 65–85, 1994.

[10] R. Eberhart and J. Kennedy, "Particle swarm optimization," in Proc. IEEE Int. Conf. Neural Netw., vol. 4, 1995, pp. 1942–1948.

[11] M. Dorigo and M. Birattari, Ant Colony Optimization. Springer, 2010.

[12] S. Mirjalili, "The ant lion optimizer," Adv. Eng. Softw., vol. 83, pp. 80–98, 2015.

[13] M. Pant, H. Zaheer, L. Garcia-Hernandez, and A. Abraham, "Differential Evolution: A review of more than two decades of research," Eng. Appl. Artif. Intell., vol. 90, p. 103479, 2020.

[14] S. Mirjalili, S. M. Mirjalili, and A. Lewis, "Grey Wolf Optimizer," Adv. Eng. Softw., vol. 69, pp. 46–61, 2014.

[15] S. Afzal, "A Review of Localization Techniques for Wireless Sensor Networks," vol. 2, no. 8, pp. 7795–7801, 2012.

[16] S. Kumar, R. Sharma, and E. R. Vans, "Localization for wireless sensor networks: A neural network approach," Int. J. Comput. Netw. Commun., vol. 8, no. 1, pp. 61–71, 2016.

[17] R. Rajakumar, J. Amudhavel, P. Dhavachelvan, and T. Vengattaraman, "GWO-LPWSN: Grey wolf optimization algorithm for node localization problem in wireless sensor networks," J. Comput. Netw. Commun., vol. 2017, pp. 1–10, 2017.

[18] S. Afzal and H. Beigy, "A localization algorithm for large scale mobile wireless sensor networks: A learning approach," J. Supercomput., vol. 69, no. 1, pp. 98–120, 2014.

[19] X. Cheng, B. Narahari, R. Simha, M. X. Cheng, and D. Liu, "Strong minimum energy topology in wireless sensor networks: NP-completeness and heuristics," IEEE Trans. Mobile Comput., vol. 2, no. 3, pp. 248–256, 2003.

[20] W. Aioffi, G. Mateus, and F. Quintao, "Optimization issues and algorithms for wireless sensor networks with mobile sink," in Int. Netw. Optim. Conf., 2007, pp. 1–6.

[21] B. Yuan, M. Orlowska, and S. Sadiq, "On the optimal robot routing problem in wireless sensor networks," IEEE Trans. Knowl. Data Eng., vol. 19, no. 9, pp. 1252–1261, 2007.

[22] W. Liu and Z. Fan, "Coverage optimization of wireless sensor networks based on chaos particle swarm algorithm," J. Comput. Appl., vol. 2, 2011.

[23] N. A. B. Ab Aziz, A. W. Mohemmed, and M. Y. Alias, "A wireless sensor network coverage optimization algorithm based on particle swarm optimization and Voronoi diagram," in Proc. Int. Conf. Netw., Sens. Control, 2009, pp. 602–607.

[24] A. Parnianifard et al., "Hybrid Metamodeling / Metaheuristic Assisted Multi-Transmitters Placement Planning," Comput., Mater. Continua, vol. 68, no. 1, pp. 569–587, 2021.

[25] A. Parnianifard, A. Sharma, and S. Chaudhary, "A new approach for data-driven surrogate modelling applied in highly nonlinear engineering functions," J. Stat. Comput. Simul., pp. 1–24, Dec. 2024.

[26] A. Parnianifard, R. Chancharoen, G. Phanomchoeng, and L. Wuttisittikulkij, "A New Approach for Low-Dimensional Constrained Engineering Design Optimization Using Design and Analysis of Simulation Experiments," Int. J. Comput. Intell. Syst., vol. 13, no. 1, pp. 1663–1678, 2020.

[27] J. Stork et al., "Open Issues in Surrogate-Assisted Optimization," in High-Performance Simulation-Based Optim., vol. 833, Springer, Cham, 2020, pp. 225–244.

[28] N. Bartoli et al., "Adaptive modeling strategy for constrained global optimization with application to aerodynamic wing design," Aerosp. Sci. Technol., vol. 90, pp. 85–102, Apr. 2019.

[29] A. Parnianifard, A. S. Azfanizam, M. K. A. Ariffin, and M. I. S. Ismail, "Trade-off in robustness, cost and performance by a multi-objective robust production optimization method," Int. J. Ind. Eng. Comput., vol. 10, no. 1, pp. 133–148, 2019.

[30] A. Parnianifard, A. S. Azfanizam, M. K. A. Ariffin, and M. I. S. Ismail, "Kriging-Assisted Robust Black-Box Simulation Optimization in Direct Speed Control of DC Motor Under Uncertainty," IEEE Trans. Magn., vol. 54, no. 7, pp. 1–10, 2018.

[31] A. Parnianifard, V. Rezaie, S. Chaudhary, M. A. Imran, and L. Wuttisittikulkij, "New Adaptive Surrogate-Based Approach Combined Swarm Optimizer Assisted Less Tuning Cost of

Authorized licensed use limited to: University of Pennsylvania. Downloaded on January 22,2026 at 04:59:01 UTC from IEEE Xplore. Restrictions apply.

Dynamic Production-Inventory Control System," IEEE Access, vol. 9, pp. 144054–144066, 2021.

[32] A. Parnianifard and A. S. Azfanizam, "Metamodel-based robust simulation-optimization assisted optimal design of multiloop integer and fractional-order PID controller," Int. J. Numer. Model., vol. 33, no. 1, p. e2679, 2020.

[33] A. Parnianifard, M. Fakhfakh, M. Kotti, A. Zemouche, and L. Wuttisittikulkij, "Robust tuning and sensitivity analysis of stochastic integer and fractional-order PID control systems: application of surrogate-based robust simulation-optimization," Int. J. Numer. Model., vol. 34, no. 2, p. e2835, 2020.

[34] A. Parnianifard, A. S. Azfanizam, M. K. A. Ariffin, and M. I. S. Ismail, "Kriging-assisted robust black-box simulation optimization in direct speed control of DC motor under uncertainty," IEEE Trans. Magn., vol. 54, no. 7, pp. 1–10, Jul. 2018.

[35] F. A. C. Viana, T. W. Simpson, V. Balabanov, and V. Toropov, "Metamodeling in Multidisciplinary Design Optimization: How Far Have We Really Come?," AIAA J., vol. 52, no. 4, pp. 670–690, 2014.

[36] A. Mohammad Nezhad and H. Mahlooji, "An artificial neural network meta-model for constrained simulation optimization," J. Oper. Res. Soc., vol. 65, no. 8, pp. 1232–1244, 2013.

[37] A. Parnianifard, A. S. Azfanizam, M. K. A. Ariffin, and M. I. S. Ismail, "An overview on robust design hybrid metamodeling: Advanced methodology in process optimization under uncertainty," Int. J. Ind. Eng. Comput., vol. 9, no. 1, pp. 1–32, 2018.

[38] A. Parnianifard, A. Azfanizam, M. Ariffin, M. Ismail, and N. Ebrahim, "Recent developments in metamodel based robust black-box simulation optimization: An overview," Decis. Sci. Lett., vol. 8, no. 1, pp. 17–44, 2019.

[39] A. Parnianifard, V. Rezaie, S. Chaudhary, M. A. Imran, and L. Wuttisittikulkij, "New adaptive surrogate-based approach combined swarm optimizer assisted less tuning cost of dynamic production-inventory control system," IEEE Access, vol. 9, pp. 144054-144066, 2021, doi: 10.1109/ACCESS.2021.3122166

[40] A. Parnianifard, A. Zemouche, R. Chancharoen, M. A. Imran, and L. Wuttisittikulkij, "Robust optimal design of FOPID controller for five bar linkage robot in a Cyber-Physical System: A new simulation-optimization approach," PLOS ONE, vol. 15, no. 11, p. e0242613, Nov. 2020.

[41] A. Parnianifard, A. S. Azfanizam, M. K. A. Ariffin, and M. I. S. Ismail, "Crossing weighted uncertainty scenarios assisted distribution-free metamodel-based robust simulation optimization," Eng. Comput., vol. 36, no. 1, pp. 139–150, 2019.

[42] A. Parnianifard, A. S. Azfanizam, M. K. A. Ariffin, and M. I. S. Ismail, "Comparative study of metamodeling and sampling design for expensive and semi-expensive simulation models under uncertainty," SIMULATION, vol. 96, no. 1, pp. 89–110, May 2019.

[43] S. Wang, X. Yang, X. Wang, and Z. Qian, "A virtual force algorithm-lévy-embedded grey wolf optimization algorithm for wireless sensor network coverage optimization," Sensors, vol. 19, no. 12, p. 2735, 2019.

[44] H. Faris, I. Aljarah, M. A. Al-Betar, and S. Mirjalili, "Grey wolf optimizer: a review of recent variants and applications," Neural Comput. Appl., vol. 30, no. 2, pp. 413–435, 2018.

[45] X. Wang, H. Gu, Y. Liu, and H. Zhang, "A two-stage RPSO-ACS based protocol: a new method for sensor network clustering and routing in mobile computing," IEEE Access, vol. 7, pp. 113141–113150, 2019.

[46] D. Wang, H. Wang, X. Ban, X. Qian, and J. Ni, "An adaptive, discrete space oriented wolf pack optimization algorithm for a movable wireless sensor network," Sensors, vol. 19, no. 19, p. 4320, 2019.

[47] A. Adulyasas, Z. Sun, and N. Wang, "Connected coverage optimization for sensor scheduling in wireless sensor networks," IEEE Sens. J., vol. 15, no. 7, pp. 3877–3892, 2015.

[48] A. P. A. S. Sushank Chaudhary, Optical and Wireless Communications Applications of Machine Learning and Artificial Intelligence. Boca Raton: CRC Press, 2025.

[49] R. Song, Z. Xu, and Y. Liu, "Wireless Sensor Network Coverage Optimization Based on Fruit Fly Algorithm," Int. J. Online Eng., vol. 14, no. 6, 2018.

[50] S. Wang, X. Yang, X. Wang, and Z. Qian, "A virtual force algorithm-Lévy-embedded grey wolf optimization algorithm for wireless sensor network coverage optimization," Sensors, vol. 19, no. 12, p. 2735, Dec. 2019, doi: 10.3390/s19122735

[51] W.-H. Liao, Y. Kao, and R.-T. Wu, "Ant colony optimization based sensor deployment protocol for wireless sensor networks," Expert Syst. Appl., vol. 38, no. 6, pp. 6599–6605, 2011.

[52] I. Vilovic, N. Burum, and Z. Sipus, "Ant colony approach in optimization of base station position," in Proc. 3rd Eur. Conf. Antennas Propag., 2009, pp. 2882–2886.

[53] C. K. Ting, C. N. Lee, H. C. Chang, and J. S. Wu, "Wireless Heterogeneous Transmitter Placement Using Multi-Objective Variable-Length Genetic Algorithm," IEEE Trans. Syst., Man, Cybern. B, vol. 39, no. 4, pp. 945–958, 2009.

[54] R. L. Haupt and D. H. Werner, Genetic Algorithms in Electromagnetics. John Wiley & Sons, 2007.

[55] Z. Yangyang, J. Chunlin, Y. Ping, L. Manlin, W. Chaojin, and W. Guangxing, "Particle swarm optimization for base station placement in mobile communication," in Proc. IEEE Int. Conf. Netw., Sens. Control, vol. 1, 2004, pp. 428-432.

[56] Z. Miao, X. Yuan, F. Zhou, X. Qiu, Y. Song, and K. Chen, "Grey wolf optimizer with an enhanced hierarchy and its application to the wireless sensor network coverage optimization problem," Appl. Soft Comput., vol. 96, p. 106602, Aug. 2020.

[57] M. Mitchell, An Introduction to Genetic Algorithms. MIT Press, 1998.

[58] M. Dorigo, M. Birattari, and T. Stutzle, "Ant colony optimization," IEEE Comput. Intell. Mag., vol. 1, no. 4, pp. 28–39, 2006.

Authorized licensed use limited to: University of Pennsylvania. Downloaded on January 22,2026 at 04:59:01 UTC from IEEE Xplore. Restrictions apply.